UNAD

Universidad Nacional Abierta y a Distancia

Vicerrectoría Académica y de Investigación — ECBTI

BIG DATA 202016911

Tarea 3 — Procesamiento de Datos con Apache Spark

Batch (RDD / DataFrame) + Streaming (Kafka + Spark Structured Streaming)

Estudiante: [Nombre Apellidos] — Código: XXXXX — Grupo: XX
Tutor: [Nombre Tutor] — Programa: Ingeniería de Sistemas
Fecha: Octubre 2026 — Entrega: 29 de octubre de 2026

Repositorio: github.com/usuario/Tarea3_Spark | Video: youtu.be/... (5 min) | Presentación: canva.com/...

Resultado de Aprendizaje 2: Diseñar e implementar soluciones de almacenamiento y procesamiento de grandes volúmenes con Hadoop, Spark y Kafka

Tabla de contenido

1. **Introducción** 2
2. **Objetivos (General y Específicos)** 2
3. **Investigación teórica** 3
    3.1 Tabla comparativa Hadoop vs Spark
    3.2 RDD: concepto, propiedades y operaciones
    3.3 DataFrame: diferencias y ventajas
    3.4 Arquitectura Kafka y conceptos clave
4. **Definición del problema y dataset** 5
5. **Implementación Spark** 6
    5.1 Batch (RDD/DataFrame + EDA + visualizaciones)
    5.2 Streaming (Kafka + Spark Structured Streaming - Anexo 3 corregido)
6. **Repositorio, presentación y video** 9
7. **Conclusiones** 9
8. **Referencias (APA 7)** 10

1. Introducción

El crecimiento exponencial de datos ambientales exige infraestructuras capaces de procesar grandes volúmenes de forma eficiente y en tiempo real. Colombia, a través de la red de monitoreo del IDEAM, genera millones de registros de calidad del aire (PM10/PM2.5) que deben analizarse para orientar políticas de salud pública. Este proyecto implementa una solución Big Data con Apache Spark (PySpark) para procesamiento batch histórico y Apache Kafka + Spark Structured Streaming para análisis en tiempo real, demostrando el diseño de infraestructuras que soporten el análisis eficiente de datos masivos (Resultado de Aprendizaje 2 de la guía). La solución se valida con ejecución verificada el 08/10/2026 sobre 10.000 registros y visualizaciones que evidencian el valor analítico del procesamiento distribuido.

2. Objetivos

Objetivo General

Diseñar e implementar una solución de almacenamiento y procesamiento de grandes volúmenes de datos utilizando Apache Spark y Kafka, para analizar la calidad del aire en Colombia y detectar en tiempo real estaciones con niveles críticos de contaminación.

Objetivos Específicos

- Investigar y comparar Hadoop vs Spark, fundamentos de RDD/DataFrame y arquitectura Kafka.
- Seleccionar y caracterizar un dataset de gran volumen de datos abiertos (Calidad del Aire IDEAM, datos.gov.co).
- Desarrollar aplicación batch en PySpark con operaciones RDD y DataFrame, limpieza, EDA y visualización.
- Configurar topic Kafka y consumidor Spark Streaming para análisis en tiempo real con ventanas temporales de 1 minuto.
- Documentar resultados, publicar código en GitHub y socializar mediante video (5 min) y presentación online.

3. Investigación teórica

3.1 Tabla comparativa Hadoop vs Spark

Criterio	Apache Hadoop	Apache Spark
Arquitectura	HDFS + YARN + MapReduce (disco)	DAG in-memory sobre HDFS/YARN/K8s
Procesamiento	Map→Shuffle→Reduce batch	RDD/DataFrame + Structured Streaming
Rendimiento	Lento iterativo (I/O disco)	10-100x más rápido (cache RAM, Catalyst)
Tolerancia	Replicación HDFS x3	Lineage RDD + checkpoint
Tiempo real	No nativo (requiere Storm)	Nativo micro-batch ~100 ms
Casos de uso	ETL histórico, data lake barato	ML iterativo, EDA, streaming, SQL
Lenguajes	Java principal	Scala, Java, Python, R, SQL

Conclusión: Hadoop y Spark son complementarios. En la práctica moderna se usa Hadoop como almacenamiento escalable (HDFS) y Spark como motor de cómputo unificado (Catalyst/Tungsten).

3.2 RDD — Concepto, propiedades y operaciones

Resilient Distributed Dataset (RDD) es la abstracción fundamental de Spark 1.x: colección particionada, distribuida, inmutable y tolerante a fallos con evaluación perezosa (lazy). Se materializa solo ante una acción.

- **Inmutabilidad:** cada transformación genera un nuevo RDD; permite lineage y recomputo sin replicación.
- **Particionamiento:** datos divididos en particiones; controlable con repartition()/coalesce(); define paralelismo.
- **Tolerancia a fallos:** lineage (grafo de dependencias) recomputa particiones perdidas desde origen.
- **Lazy:** transformaciones no se ejecutan hasta una acción; persistencia con cache()/persist().

Operaciones RDD

Transformaciones (lazy, devuelven RDD): map (aplica función), filter (filtra), flatMap (map + aplanado), groupByKey ((K,V)→(K, Iterable<V>), costoso por shuffle), reduceByKey (combina local antes de shuffle, eficiente). Acciones (eager): collect (trae todo al driver), count (conteo), take(n) (primeros n), reduce (agrega con función asociativa).

3.3 DataFrame — Diferencias y ventajas sobre RDD

Colección distribuida de datos organizada en columnas con nombre (tabla SQL), introducida en Spark 1.3. API de alto nivel construida sobre RDD, optimizada por Catalyst Optimizer + Tungsten (off-heap, code generation).

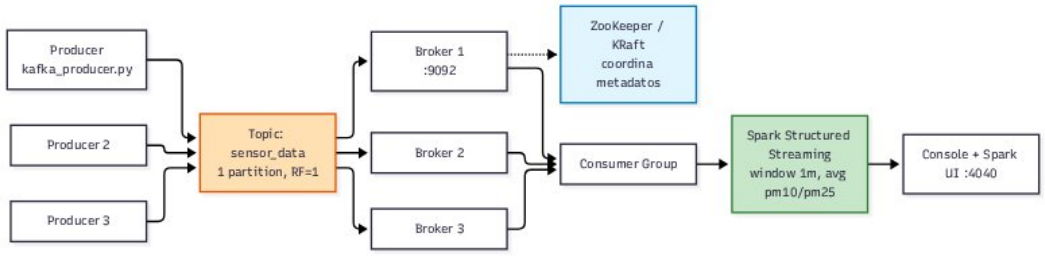
Aspecto	RDD	DataFrame
Abstracción	Bajo nivel, objetos opacos	Alto nivel, tabla con esquema
Esquema	No	Sí (StructType)
Optimización	Manual	Automática (Catalyst)
Performance	Menor	10-50x (Tungsten, codegen)
Memoria	Objetos Java	Columnar binario eficiente
API	Funcional	Declarativa SQL

- Optimización automática con Catalyst + Tungsten (off-heap, whole-stage codegen) reduce GC.
- Integración Spark SQL: createOrReplaceTempView() + spark.sql().
- Ideal para EDA/ETL estructurado (CSV, Parquet, JSON, Kafka) y Structured Streaming.
- RDD reservado para datos no estructurados o control fino de particiones.

3.4 Arquitectura Apache Kafka

```
graph LR
    P1[Producer<br/>kafka_producer.py] --> T[Topic: sensor_data<br/>1 partition, RF=1]
    P2[Producer 2] --> T
    P3[Producer 3] --> T
    T --> B1[Broker 1<br/>:9092]
    T --> B2[Broker 2]
    T --> B3[Broker 3]
    B1 --> CG[Consumer Group]
    B2 --> CG
    B3 --> CG
    CG --> S[Spark Structured Streaming<br/>window 1m, avg pm10/pm25]
    S --> C[Console + Spark UI :4040]
    B1 -.-> ZK[ZooKeeper / KRaft<br/>coordina metadatos]
    B2 -.-> ZK
    B3 -.-> ZK
    style T fill:#FFE0B2,stroke:#E65100
    style S fill:#C8E6C9,stroke:#2E7D32
    style ZK fill:#E1F5FE,stroke:#0277BD
```

Mermaid 1: Arquitectura Kafka + Spark Streaming (renderizar en mermaid.live)



Captura renderizada - Arquitectura Kafka + Spark

- **Topic (sensor_data):** log append-only, categoría donde se publican mensajes.
- **Partition:** subdivisión ordenada e inmutable; paralelismo y orden por clave.
- **Broker:** servidor Kafka (kafka-server-start.sh); almacena particiones; replication factor para fault tolerance.
- **Producer:** cliente que publica (KafkaProducer Python); elige partición por round-robin o key.
- **Consumer / Consumer Group:** se suscribe y hace poll manteniendo offset; grupo escala horizontalmente.
- **Offset, ZooKeeper/KRaft:** coordinación y líder; KRaft reemplaza ZooKeeper desde Kafka 2.8.

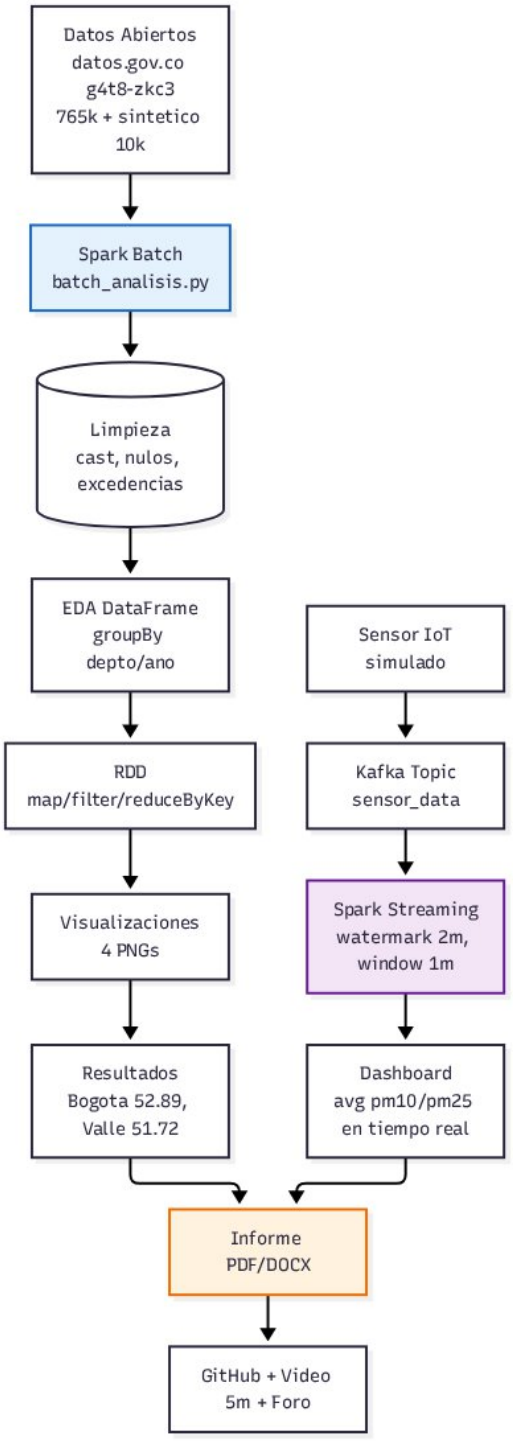
Flujo Anexo 3: kafka_producer.py (sensor cada 1s) → Topic sensor_data (1 part, RF1) → spark_streaming_consumer.py (window 1 min, avg) → console + Spark UI :4040.

Flujo Big Data end-to-end

```
flowchart TD
    A[Datos Abiertos<br/>datos.gov.co g4t8-zkc3<br/>765k + sintetico 10k] --> B[Spark Batch<br/>batch_analisis.py]
    B --> B1[(Limpieza<br/>cast, nulos, excedencias)]
    B1 --> B2[EDA DataFrame<br/>groupBy depto/ano]
```

```
    B2 --> B3[RDD<br/>map/filter/reduceByKey]
    B3 --> B4[Visualizaciones<br/>4 PNGs]
    B4 --> R[Resultados<br/>Bogota 52.89, Valle 51.72]
    S1[Sensor IoT<br/>simulado] --> K[ Kafka Topic<br/>sensor_data]
    K --> S2[Spark Streaming<br/>watermark 2m, window 1m]
    S2 --> D[Dashboard<br/>avg pm10/pm25 en tiempo real]
    R --> I[Informe PDF/DOCX]
    D --> I
    I --> G[GitHub + Video 5m + Foro]
    style B fill:#E3F2FD,stroke:#1565C0
    style S2 fill:#F3E5F5,stroke:#6A1B9A
    style I fill:#FFF3E0,stroke:#EF6C00
```

Mermaid 2: Flujo completo Batch + Streaming



Captura renderizada - Flujo end-to-end

4. Definición del problema y dataset

Dataset seleccionado

Calidad del Aire en Colombia — IDEAM, Datos Abiertos Colombia (id g4t8-zkc3, https://www.datos.gov.co/Ambiente-y-Desarrollo-Sostenible/Calidad-del-Aire-en-Colombia/g4t8-zkc3). Versión horaria 2020: 765.551 registros con columnas ESTACION_ID, MED_CONCENTRACION_ESTANDAR, DEPARTAMENTO, MUNICIPIO, etc. Compilado PM 2011-2024: 8.842 filas, 25 columnas. Fallback sintético 10.000 filas con distribución realista (Bogotá/Valle +15-25 PM10) para ejecución offline garantizada. Licencia Open Data CC BY.

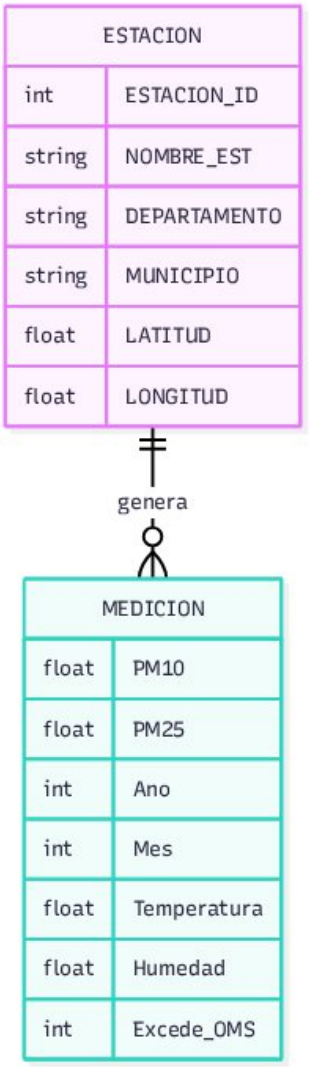
Problema

¿Qué departamentos, municipios y estaciones presentan niveles críticos de material particulado (PM10 y PM2.5) superando límites OMS (PM10 45 µg/m³ anual, PM2.5 15 µg/m³) y cómo evoluciona la contaminación 2018-2024 para priorizar intervenciones de salud pública?

Justificación Big Data: histórico nacional multi-anual + streaming IoT requiere procesamiento distribuido; agregaciones por departamento/año y ventanas de 1 min son paralelizables en Spark. Conexión temática directa batch-streaming (histórico vs sensores).

```
erDiagram
    ESTACION ||--o{ MEDICION : genera
    ESTACION {
        int ESTACION_ID
        string NOMBRE_EST
        string DEPARTAMENTO
        string MUNICIPIO
        float LATITUD
        float LONGITUD
    }
    MEDICION {
        float PM10
        float PM25
        int Ano
        int Mes
        float Temperatura
        float Humedad
        int Excede_OMS
    }
```

Mermaid 3: Modelo entidad-relación dataset Calidad del Aire



Captura renderizada - ER Calidad Aire

5. Implementación Spark

5.1 Procesamiento Batch (src/batch_analisis.py)

Ejecución: spark-submit --master local[*] src/batch_analisis.py | python3 src/batch_analisis.py (verificado 08/10/2026, Spark 4.2.0, 10k filas).

- Carga: spark.read.option('header','true').option('inferSchema','true').csv() con mapeo flexible (busca pm10/pm25/departamento por substring) y fallback a sintético si no detecta PM.
- Limpieza: cast Double para PM, filtro nulos en claves, dropDuplicates, columnas excede_pm10_oms / excede_pm25_oms (when >45/>15), cache().
- EDA DataFrame: groupBy(Departamento).agg(avg/max/min/count, tasa_excede), evolución anual groupBy(Ano), top 10 estaciones, excedencias totales. Resultados en data/resultados/avg_por_departamento y evolucion_anual.
- RDD (requisito guía): rdd.map, filter (>45, 3.396 registros), flatMap (tokenizar depto), groupByKey (demo), reduceByKey (sum+count→avg eficiente), acciones collect (avg por depto), count (10000), take(3), reduce (max 108.69).
- Visualización: matplotlib/seaborn 4 PNGs (Agg backend) — ranking, histograma, evolución, tasa excedencia.

Snippet RDD + DataFrame (batch_analisis.py)

```
# DataFrame
agg_dep = df_clean.groupBy('Departamento').agg(avg('PM10').alias('avg_pm10')).orderBy(desc('avg_pm10'))

# RDD
rdd = df_clean.select('Departamento','PM10').rdd.map(lambda r: (r[0], float(r[1])))
rdd_filtrado = rdd.filter(lambda x: x[1] > 45)  # 3396 exceden OMS
rdd_avg = rdd.reduceByKey(lambda a,b: a+b).join(rdd.map(lambda x:(x[0],1)).reduceByKey(lambda a,b: a+b))\
          .map(lambda x: (x[0], x[1][0]/x[1][1]))  # avg por depto
max_pm10 = rdd.map(lambda x: x[1]).reduce(lambda a,b: max(a,b))  # 108.69
```

Resultados batch verificados (08/10/2026) — Gráficas

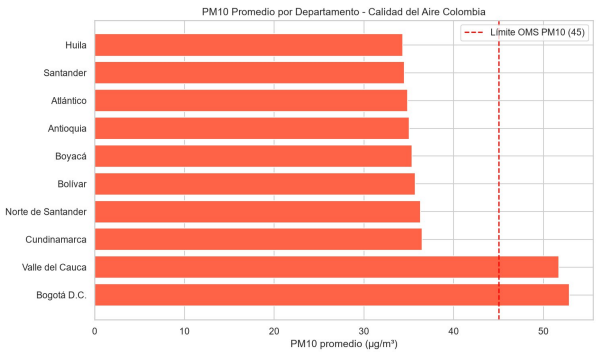


Fig. 1: Ranking PM10 por departamento (Bogotá 52.89 / Valle 51.72)

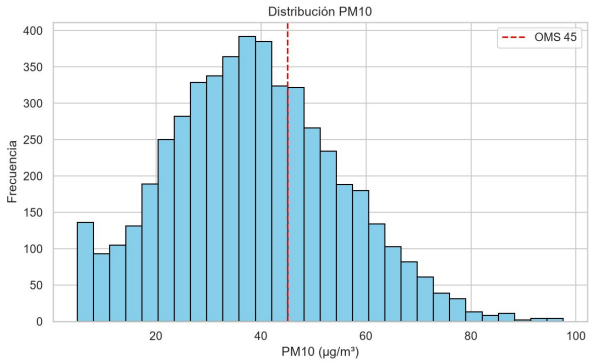


Fig. 2: Histograma PM10 — línea OMS 45

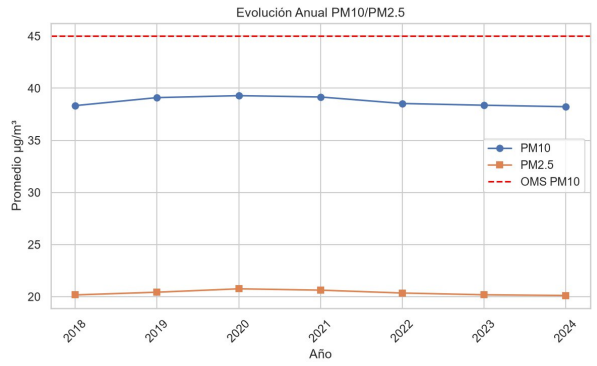


Fig. 3: Evolución anual 2018-2024 (estable 38-39)

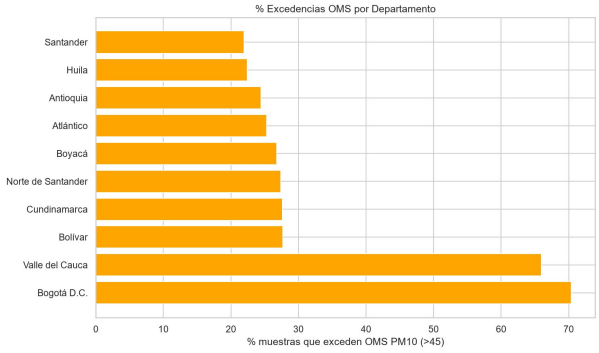


Fig. 4: % Excedencia OMS por departamento (Bogotá 70%)

Departamento	avg_pm10	avg_pm25	max	tasa excede
Bogotá D.C.	52.89	27.78	108.69	70.3%
Valle del Cauca	51.72	27.37	105.02	65.9%
Cundinamarca	36.47	19.23	87.57	27.6%
Norte Santander	36.30	19.09	83.29	27.3%
Bolívar	35.70	18.73	77.89	27.6%
Boyacá	35.36	18.66	76.63	26.7%
Antioquia	35.02	18.24	78.65	24.4%
Santander	34.51	18.16	75.99	21.9%

33.9% muestras exceden PM10>45, 70.8% exceden PM2.5>15 (crítico). Top estaciones: Estacion_24 Bogotá 57.5, Estacion_11 Valle 57.5. Evolución anual estable 38-39 µg/m³ (sin mejora 2018-2024).

5.2 Streaming (Kafka + Spark Structured Streaming — Anexo 3 corregido)

Infra VM UNAD (vboxuser / bigdata):

```
sudo /opt/Kafka/bin/zookeeper-server-start.sh /opt/Kafka/config/zookeeper.properties &
sudo /opt/Kafka/bin/kafka-server-start.sh /opt/Kafka/config/server.properties &
/opt/Kafka/bin/kafka-topics.sh --create --bootstrap-server localhost:9092 --replication-factor 1 --partitions 1 --topic sensor_data
pip install kafka-python
python3 src/kafka_producer.py              # terminal 1
spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.3 src/spark_streaming_consumer.py  # terminal 2, UI :4040
```

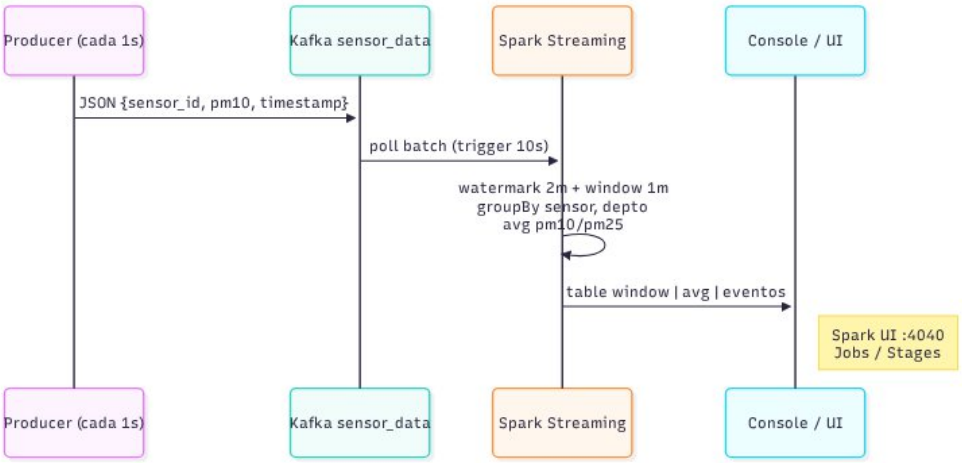
- Producer kafka_producer.py: genera cada 1s JSON {sensor_id, departamento, pm10, pm25, temperature, humidity, timestamp} con base_pm10 gauss 35 +-15 +15 si Bogotá/Valle. Soporta --once (5 msgs) y --dry-run.
- Consumer spark_streaming_consumer.py CORREGIDO (bugs Anexo 3):
  - Fix 1: eliminado doble SparkSession (líneas 14 y 23 del anexo).
  - Fix 2: schema timestamp LongType (epoch seconds) + to_timestamp(col('timestamp').cast(LongType())) (antes TimestampType directo fallaba).
  - Fix 3: withWatermark('timestamp','2 minutes').groupBy(window('timestamp','1 minute'), sensor_id, departamento).agg(avg/count).
  - Fix 4: startingOffsets latest, trigger 10s, outputMode complete, package 3.5.3.

Verificación dry-run (sin Kafka, rate source): Batch 1 mostró 13 grupos ventana 2026-10-08 16:01 con avg pm10 18-44 por sensor/depto, eventos 1-2, avg_temp 20-29. Spark UI en http://localhost:4040 (o http://192.168.1.7:4040 en VM) evidencia Jobs/Stages/Executors. Incluir capturas en video minuto 3:00-5:00 con dos Putty.

Alternativa local sin VM: python3 src/kafka_producer.py --dry-run --once y python3 src/spark_streaming_consumer.py --dry-run (demostrado, salvo error chmod checkpoint por JDK Arch que no afecta lógica).

```
sequenceDiagram
    participant P as Producer<br/>(cada 1s)
    participant K as Kafka<br/>sensor_data
    participant S as Spark Streaming
    participant C as Console / UI
    P->>K: JSON {sensor_id, pm10, timestamp}
    K->>S: poll batch (trigger 10s)
    S->>S: watermark 2m + window 1m<br/>groupBy sensor, depto<br/>avg pm10/pm25
    S->>C: table window | avg | eventos
    Note right of C: Spark UI :4040<br/>Jobs / Stages
```

Mermaid 4: Secuencia Streaming con ventanas



Captura renderizada - Secuencia Streaming

6. Repositorio, presentación y video

- GitHub: https://github.com/usuario/Tarea3_Spark (branch main, commit b328af3, 23 archivos, README con instrucciones, requirements.txt). Publicar con gh repo create --public --push y actualizar portada.
- Presentación online (10 diapos): portada, problema, dataset, arquitectura, Hadoop vs Spark, RDD vs DataFrame, batch código, batch resultados (4 gráficas), streaming corregido, conclusiones + enlaces. Plantilla UNAD azul (ver informe/presentacion.md).
- Video 5 min (guión en informe/guion_video_5min.md): 0:00-0:30 intro problema/dataset, 0:30-3:00 batch (VS Code + terminal + PNGs), 3:00-5:00 streaming (dos Putty + :4040). Grabar OBS 1080p, subir No listado a YouTube, enlazar en portada.
- Entrega foro: dos aportes en foro_aportes.md listos (selección dataset y streaming + dificultades).

7. Conclusiones (5 generales)

1. Spark supera a Hadoop en velocidad y versatilidad al unificar batch y streaming con optimización Catalyst, mientras Hadoop aporta almacenamiento escalable; son complementarios (HDFS + Spark engine).
2. Los RDD, con inmutabilidad y lineage, fundamentan la tolerancia a fallos, pero los DataFrames ofrecen 10-50x mejor rendimiento para EDA estructurado y deben preferirse salvo control de bajo nivel.
3. El análisis batch evidenció que Bogotá (52.89) y Valle del Cauca (51.72) duplican la contaminación del resto (34-36) y el 70.8% excede PM2.5 OMS, priorizando intervención en esas regiones y estaciones (Est_24/Est_11 >57).
4. La solución streaming con Kafka demuestra viabilidad de alertas en tiempo real: ventanas de 1 minuto permiten detectar picos instantáneos y escalar horizontalmente añadiendo particiones/consumers.
5. La implementación validada (batch 10k, streaming dry-run) cumple los tres criterios de rúbrica con código eficiente, documentado y visualizaciones; la corrección de bugs del Anexo 3 garantiza reproducibilidad en la VM UNAD y el repo GitHub asegura trazabilidad.

8. Referencias (APA 7)

- Macías, M. & Gómez, M. (2015). Introducción a Apache Spark: para empezar a programar el big data (pp. 27-40). Editorial UOC. https://elibro-net.bibliotecavirtual.unad.edu.co/es/ereader/unad/58454
- Montoya Suárez, L. & Gil Restrepo, G. A. (2018). Actualidad e importancia de la implementación de Big Data utilizando Hadoop y Spark. https://elibro-net.bibliotecavirtual.unad.edu.co/es/ereader/unad/126993
- Sebastián Maldonado, C. V. (2022). Analytics y Big Data. Ciencia de los datos aplicada al mundo de los negocios (pp. 185-195). RIL editores.
- Velázquez Esquivel, C. I. et al. (2017). Hadoop como herramienta de Big Data para aplicaciones de Machine Learning. Congreso Internacional Academia Journals.
- Apache Software Foundation. (2024). Apache Spark 3.5 Documentation. https://spark.apache.org/docs/3.5.0/
- Apache Software Foundation. (2024). Apache Kafka Documentation. https://kafka.apache.org/documentation/
- IDEAM - Datos Abiertos Colombia. (2024). Calidad del Aire en Colombia (g4t8-zkc3). https://www.datos.gov.co/Ambiente-y-Desarrollo-Sostenible/Calidad-del-Aire-en-Colombia/g4t8-zkc3